

PARTICIPANT STUDY MATERIAL

Agentic AI for Teaching & Academic Processes

From simple flows, to good prompts, to agents you write yourself

Faculty Development Workshop

Symbiosis Institute of Technology, Pune

Facilitator: Mr. Dinesh Wadhwani

Day 2: 10 September 2026, 1.30 pm – 4.00 pm

Day 3: 11 September 2026, 9.30 am – 4.00 pm

Track A: Apify & Google Gemini

Track B: n8n (optional)

Companion volume: Lab Manual & Test Set

How to use this book

This is the study material for a one-and-a-half day workshop on using AI agents in teaching and academic work. It is written to be read **during** the sessions and kept **afterwards** as a reference.

The path we will take

We build in a deliberate order, and the order matters more than any individual tool:

Simple flows → good prompts → agents.

Most workshops on this subject begin with the word "agent" and never recover. Participants leave impressed but unable to build anything, because they were shown the most complex thing first. We do the reverse. By the time we use the word "agent" in a hands-on session, you will already have built something that works, and you will understand precisely which problem the agent is solving.

Structure

Part	Sessions	What you will be able to do
Part 1 — Foundations	Day 2, afternoon	Write prompts that produce usable academic output; build and run a working automated flow
Part 2 — Agents	Day 3, full day	Build agents that use tools, check each other's work, and run against your own course material

Each chapter ends with a **Lab** reference. The full step-by-step instructions for every lab are in the companion volume, *Lab Manual & Test Set*. This book explains **why**; that book explains **how**.

Conventions

CONCEPT

Grey boxes explain an idea you need before the next step.

WATCH OUT

Red boxes flag things that commonly go wrong. Please read these ones.

IN PRACTICE

Green boxes contain advice from actually running these things in an academic setting.

Text in `this typeface` is something you type or click exactly as written.

An honest word before we start

Everything in this book runs on free tiers. That is a real constraint and it shapes what we build. You will not leave here with a production system serving two thousand students. You will leave with working prototypes for your own teaching, which is a much more useful thing to have, and a clear understanding of what it would take to go further.

You will also be shown things failing. Deliberately, and more than once. A workshop that only demonstrates successes teaches you that this technology works, which is half a truth and the more dangerous half.

Chapter 1 — What actually changed

1.1 The question worth asking

Faculty have been using AI tools for a while now. You have asked a chatbot to rephrase a paragraph, generate quiz questions, or explain a concept differently. It works, it saves some time, and it stops there.

The reason it stops there is worth naming precisely, because it defines everything we will build.

When you use a chatbot, you are the runtime.

You decide what to ask. You paste in the material. You read the output, judge it, decide the next step, and ask again. The intelligence is in the model, but the *process* is in you. Close the tab and nothing persists. Do the same task next week and you do the whole thing again from scratch.

Every technique in this workshop is an answer to one question: **how much of that process can move out of your head and into something that runs on its own?**

1.2 Three tiers

There are exactly three levels, and confusing them is the single most common source of wasted effort in this field.

	Chatbot	Workflow	Agent
Who decides the steps	You, live	The designer, in advance	The model, at runtime
Runs without you	No	Yes	Yes
Same input, same output	Roughly	Exactly	Not necessarily
Can handle the unexpected	You handle it	No — it breaks	Sometimes
Cost to build	Zero	Low	Moderate
Cost to trust	You check everything	Test once, then reliable	Ongoing supervision
Example	Asking Gemini to draft five MCQs	A flow that reads a marks CSV, computes attainment, and emails a report every Friday	A grader that reads a submission, decides which rubric criteria apply, scores each, and revises after a second agent objects

Read the row "**Cost to trust**" again. It is the row everyone skips and the row that decides whether any of this survives contact with real academic work.

THE DEFINITION TO HOLD ON TO

A **workflow** is a fixed sequence of steps that you designed. It does exactly what you told it to, every time.

An **agent** is a model that can choose an action, take it, look at the result, and decide what to do next — without you in the loop for each step.

The difference is not intelligence. It is *who chooses the next step*.

1.3 The advice nobody gives you

Here is the counterintuitive part, and it will save you a great deal of time over the next year:

Most academic tasks do not need an agent. They need a workflow.

Computing CO attainment from a marks sheet is a fixed procedure. The steps never change. You do not want a model deciding creatively how to calculate attainment this Tuesday — you want the same correct calculation every single time. That is a workflow, and building it as an "agent" makes it slower, more expensive, and less reliable.

Ask yourself one question:

Do the steps change depending on what I find?

If no, build a workflow. If yes, you may need an agent.

Academic task	Steps change?	Build it as
Compute CO/PO attainment from marks	No — fixed formula	Workflow
Generate a question paper to a fixed blueprint	No — fixed structure	Workflow with an AI step
Grade essays against a rubric	Slightly — depends on what the student wrote	Workflow with an AI step, plus review
Answer a student's doubt on your course	Yes — depends entirely on the question	Agent
Investigate why a cohort underperformed	Yes — each finding suggests the next check	Agent
Draft NBA documentation from scattered records	Yes — depends what records exist	Agent

Notice that three of six are workflows. In real departmental use the ratio is more lopsided still. The industry talks about agents because agents are interesting. Your Friday afternoon is saved by workflows.

1.4 Why two platforms

We build on **Apify** and, where the machines allow it, **n8n**. Not to be thorough, but because the comparison teaches something neither does alone.

	Apify	n8n
What it is	Cloud platform. You write code.	Local tool. You drag boxes.
The agent loop	You write it — you can read it	Hidden inside a node
Install needed	None. Browser only.	Node.js and npm on the machine
Iteration speed	Slower — code changes need a rebuild	Instant
Where your work lives	Your own account	That particular lab machine
Runs on a schedule	Yes, via API	Yes, natively

Apify is the spine of this workshop, for two reasons.

Nothing needs installing. Everything runs in a browser tab. Your work sits in your own account and comes home with you rather than living on a lab machine that gets reimaged.

You write the agent loop yourself. This matters more than it sounds. n8n's Agent node, CrewAI, LangGraph — all of them hide the loop. You configure an agent, it works, and you have learned an import statement. The loop you will write in Chapter 6 is about sixty lines. Once you have read it, every agent framework you meet afterwards is a matter of looking up method names.

n8n earns its place as the counterpoint. It is faster to assemble, it shows you the data at every step, and for a scheduled job that connects to your email and a spreadsheet it is genuinely the better tool. You will build the same attainment agent both ways and decide for yourself.

N8N MAY NOT RUN

n8n requires software installed on the lab machines. If that did not happen in time, the n8n exercises are dropped and nothing essential is lost.

Every concept in this workshop is taught on Apify. The n8n labs are comparison exercises.

1.5 Where Apify Actors come in

On Apify, a program is called an **Actor**. It takes structured input, does something, and writes structured output. That is the entire model.

An Actor can be almost anything. In this workshop yours will be:

- A grade calculator with no AI at all
- A feedback generator that calls Gemini once per student
- An agent that loops, calls tools, and decides when it is done
- A three-stage grading pipeline
- A tool that fetches and cleans a web page

Actors also give an agent **reach** — the ability to fetch information that is not in its training data and not in your files. A current syllabus from an affiliated university's site, recent papers for a literature assistant, documentation for a tool you teach.

You will **build** Actors rather than renting them from Apify's Store. That is a cost decision, and Chapter 9 shows the arithmetic.

1.6 What we are building

Everything across both days circles four use cases drawn from work you already do:

#	Build	What it teaches
1	Question paper generator, CO and Bloom's mapped	Prompting and grounding
2	CO-PO attainment analyser	Tool use — why an agent must calculate rather than guess
3	Rubric grader with a moderator	Multi-agent review
4	Course doubt-solving agent for students	Retrieval and guardrails

These are not illustrations. By Thursday evening each will be running against your own subject.

CHAPTER 1 — CHECK YOURSELF

- [] I can state the difference between a workflow and an agent in one sentence
- [] I can name one task from my own teaching that is a workflow, not an agent
- [] I understand why "cost to trust" is the row that matters

Chapter 2 — Prompts that survive contact

2.1 Why this chapter comes before any tool

Every agent you build for the rest of this workshop is, at its core, a prompt with plumbing around it. If the prompt is vague, adding tools and memory and multi-agent review produces vague output more elaborately and at greater expense.

This chapter is therefore the highest-leverage hour of the entire workshop. Please do not treat it as preamble.

2.2 A realistic starting point

Here is a prompt very close to what most faculty actually type:

```
Generate 5 questions on Database Normalization for my exam.
```

The output will be five questions. They will look plausible. They will also, typically:

- All sit at the same cognitive level, usually recall
- Carry no marks allocation
- Map to no course outcome
- Repeat one another with different wording
- Include one question on a topic you do not teach
- Be pitched at whatever difficulty the model assumed

None of that is a model failure. The model was told almost nothing and filled the gaps with averages. **Vague prompts do not produce wrong answers; they produce average ones.** For teaching, average is the problem.

2.3 The RCTFG pattern

Five components. If a prompt is not working, one of them is missing. In four years of working with this the fault is almost always F or G.

	Component	The question it answers
R	Role	Who is the model being?
C	Context	What material must it work from?
T	Task	What exactly must it produce?
F	Format	What shape must the output take?
G	Guardrails	What must it never do?

R — Role

Sets vocabulary, assumed audience, and depth.

```
You are an experienced professor of Computer Engineering setting
internal assessment papers for a second-year B.Tech course at an
Indian engineering institute affiliated to a state university.
```

Note the specificity. "You are a teacher" does almost nothing. "Second-year B.Tech, Indian institute, internal assessment" pins down difficulty, format expectations, and terminology all at once.

C — Context

The material it must work from. This is what separates a generic question generator from *your* question generator.

```
COURSE: Database Management Systems (CS2xx), Semester 4

SYLLABUS UNIT 3 – Normalization:
Functional dependencies; Armstrong's axioms; closure of attribute
sets; 1NF, 2NF, 3NF, BCNF; lossless-join and dependency-preserving
decomposition; multivalued dependencies and 4NF.

COURSE OUTCOMES:
C03: Apply normalization techniques to eliminate redundancy and
      anomalies in relational schema design.
C04: Analyse a given relational schema and decompose it while
      preserving dependencies.

NOT TAUGHT IN THIS COURSE: 5NF, domain-key normal form,
temporal databases.
```

THE LINE THAT DOES THE MOST WORK

NOT TAUGHT IN THIS COURSE: ... is worth more than any other single line in the prompt.

Models are trained on the whole of the internet's database material and will drift toward the mean of it. Telling it what is out of scope is far more effective than hoping it stays in scope. Every prompt you write for a course should carry this line.

T — Task

Specific enough that two different people would produce comparable output from it.

```
Generate a 20-mark internal assessment covering C03 and C04.
```

```
Structure:
```

- 2 questions at 3 marks (Understand level)
- 2 questions at 4 marks (Apply level)
- 1 question at 6 marks (Analyse level)

```
At least one Apply-level and the Analyse-level question must give
a concrete relation with sample functional dependencies for the
student to work on. Do not ask students to merely define or list.
```

F — Format

Under-specified format is the most common reason output is unusable. You want to paste it into a question paper template, not spend twenty minutes reformatting.

```
Output a Markdown table with exactly these columns:
```

```
Q.No | Question | Marks | C0 | Bloom's Level | Expected Answer Outline
```

```
After the table, add a section headed "Marking Scheme" with a
mark-by-mark breakdown for each question.
```

```
Do not include any preamble, commentary, or explanation of your
approach. Output the table and the marking scheme only.
```

That last sentence saves you roughly ninety seconds every single run.

G — Guardrails

What it must refuse. Without these, the model always produces *something*, which is exactly the failure mode you cannot detect by skimming.

CONSTRAINTS:

- Use only topics listed in the syllabus section above. If asked for a topic not listed, state clearly that it is out of syllabus and generate nothing for it.
- Tag every question with the CO it assesses. If a question does not clearly map to a listed CO, discard it and generate another.
- Bloom's level tags must match the verb actually used in the question. Do not label a "define" question as Analyse.
- Marks must total exactly 20.

2.4 The complete prompt

Assembled, this is what a working academic prompt looks like. It is long. Length is not the point — *specificity* is — but specificity takes words.

You are an experienced professor of Computer Engineering setting internal assessment papers for a second-year B.Tech course at an Indian engineering institute affiliated to a state university.

COURSE: Database Management Systems (CS2xx), Semester 4

SYLLABUS UNIT 3 – Normalization:

Functional dependencies; Armstrong's axioms; closure of attribute sets; 1NF, 2NF, 3NF, BCNF; lossless-join and dependency-preserving decomposition; multivalued dependencies and 4NF.

COURSE OUTCOMES:

C03: Apply normalization techniques to eliminate redundancy and anomalies in relational schema design.

C04: Analyse a given relational schema and decompose it while preserving dependencies.

NOT TAUGHT IN THIS COURSE: 5NF, domain-key normal form, temporal databases.

TASK:

Generate a 20-mark internal assessment covering C03 and C04.

- 2 questions at 3 marks (Understand level)
- 2 questions at 4 marks (Apply level)
- 1 question at 6 marks (Analyse level)

At least one Apply-level and the Analyse-level question must give a concrete relation with sample functional dependencies for the student to work on. Do not ask students to merely define or list.

OUTPUT FORMAT:

A Markdown table with columns:

Q.No | Question | Marks | CO | Bloom's Level | Expected Answer Outline

Followed by a "Marking Scheme" section with mark-by-mark breakdown.

No preamble or commentary.

CONSTRAINTS:

- Use only syllabus topics listed above. If asked for anything outside it, say so and generate nothing for that topic.
- Tag every question with its CO. Discard any question that does not clearly map to a listed CO.
- Bloom's tags must match the verb actually used in the question.
- Marks must total exactly 20.

WRITE IT ONCE

This looks like a lot of typing. It is, once. From here it becomes a template: swap the syllabus block and the CO block and you have the same instrument for any unit of any course you teach.

In Lab 2 you will save this as a reusable asset so you never retype it again.

2.5 Four techniques worth knowing

Show one example

A single worked exemplar shapes output more strongly than three paragraphs of description. Add to the prompt:

EXAMPLE OF THE STYLE REQUIRED:

Q. Consider the relation $R(A, B, C, D, E)$ with functional dependencies: $A \rightarrow BC$, $CD \rightarrow E$, $B \rightarrow D$, $E \rightarrow A$. Determine whether R is in BCNF. If not, decompose it into BCNF and state whether your decomposition is dependency-preserving. [6 marks] [C04] [Analyse]

Generate questions of comparable specificity and rigour.

This is the fastest single fix for output that is technically correct but pitched wrongly.

Make it show its reasoning

Add:

For each question, add one sentence explaining why you assigned that C0 and that Bloom's level.

Two effects. It slightly improves the tagging. More importantly, **it lets you audit the tagging in seconds**. A wrong CO is nearly invisible in a table; a wrong justification is glaringly obvious.

This is the single most useful habit in this chapter.

Anchor Bloom's to verbs

Models inflate Bloom's levels — everything becomes "Analyse" because it sounds better. Anchor it:

BLOOM'S LEVEL DEFINITIONS FOR TAGGING:

Remember - define, list, state, name
Understand- explain, describe, illustrate, classify
Apply - solve, compute, demonstrate, normalize (a given relation)
Analyse - compare, differentiate, decompose, examine trade-offs
Evaluate - justify, critique, assess, defend a design choice
Create - design, construct, formulate a new schema

Tag based on the verb the question actually requires the student to perform, not on the topic's difficulty.

That last sentence corrects the specific error models make: treating a hard topic as a high Bloom's level. Asking a student to *define* BCNF is Remember, however advanced BCNF is.

Iterate on the prompt, not the output

When output is wrong, the instinct is to fix the output. Resist it. Fix the *prompt* and regenerate.

Editing output helps you once. Editing the prompt helps you every time you use it for the rest of the semester.

2.6 What prompting cannot fix

Necessary honesty before we build anything on this foundation.

It cannot make the model reliable at arithmetic. No prompt makes an LLM trustworthy at computing attainment across sixty rows of marks. This is not a prompting problem and has a proper solution — tools, Chapter 7.

It cannot make it know your material. No amount of role-setting makes it aware of what you covered in class on 14 August. That is grounding, Chapter 10.

It cannot make it consistent. Run the same prompt three times, get three variations. For question generation that is a feature. For grading it is a serious problem, and it drives the design in Chapter 8.

It cannot make it admit ignorance reliably. Guardrails help substantially. They do not eliminate confident invention. **Verification remains your job.**

THE SENTENCE TO CARRY OUT OF THIS CHAPTER

It is fluent before it is correct.

Fluency and correctness are separate properties, and this technology delivers the first far more reliably than the second. Your judgement is not being replaced — it is being relocated, from producing material to checking it. That is still a large saving. It is a different job, and pretending otherwise is how institutions get burned.

CHAPTER 2 — CHECK YOURSELF

- [] I can name the five parts of RCTFG without looking
- [] I know which two are most often missing
- [] I have written the "NOT TAUGHT IN THIS COURSE" line for one of my own units
- [] I understand why asking for a justification is an auditing tool

→ **Lab 1: Prompt engineering for academic output** → **Lab 2: Building a reusable question paper generator**

Chapter 3 — Your first Actor

3.1 Three sections, always

An Actor has exactly three parts, and every Actor you build in this workshop has this shape.

```
1. READ THE INPUT    const input = await Actor.getInput();
2. DO THE WORK      ...ordinary code...
3. WRITE THE OUTPUT  await Actor.pushData(results);
```

Only the middle section changes. A grade calculator, an agent that loops for five turns, a three-stage grading pipeline — all of them are input, work, output.

That is genuinely all there is to the platform model. Everything else is detail.

3.2 The four files

When you create an Actor from the Node.js template you get a file tree. Four files matter, and you will only ever edit two of them.

File	What it does	You edit it?
<code>src/main.js</code>	Your code	Yes, constantly
<code>.actor/input_schema.json</code>	Defines the input form	Yes
<code>.actor/actor.json</code>	Actor metadata	No
<code>package.json</code>	Dependencies	No

3.3 The input schema is the interface

`input_schema.json` deserves more attention than its name suggests. Everything you declare there appears as a form field on the Actor's Input tab, which people fill in before running.

That sounds like a minor convenience. It is the single most important design decision on this platform, for one reason:

THE FAST PATH AND THE SLOW PATH

Change	Rebuild needed?	Time
Anything in the input schema	No	About ten seconds
Anything in <code>main.js</code>	Yes	One to two minutes

Editing code and rebuilding takes one to two minutes. Editing an input field and clicking Start takes ten.

So put the things you tweak often into the input schema. Above all, put your **prompts** there. A prompt hardcoded in `main.js` means a two-minute wait for every wording change. A prompt in an input field means you can try six versions in two minutes.

That single decision is the difference between this platform being pleasant to work on and being unbearable.

Ask, of every value in your code: *am I going to want to change this without changing the logic?* If yes, it belongs in the schema.

3.4 Memory is what you pay for

Apify bills in **Compute Units**. One CU is one gigabyte of RAM running for one hour.

At 512 MB for thirty seconds, a run costs a fraction of a cent. At 4 GB it costs eight times as much for identical work.

Set memory to the lowest value that works, every time. This is the biggest lever you have on the \$5 monthly credit, and it is set on the Input tab before you click Start, not in code.

3.5 Reading the log

Every `console.log` in your code appears in the Log tab while the run is in progress. This is your main debugging surface.

Log generously. In particular, log at the boundaries: what came in, what each step produced, what went out.

THE DEBUGGING RULE

When an Actor misbehaves, do not re-read your logic first. Read the log top to bottom and find **the first line that is not what you expected**.

That line is your bug. Everything after it is a symptom.

3.6 The same thing in n8n

If Track B is running you will build the identical logic as an n8n workflow, and one idea there is worth knowing in advance because it causes most beginner confusion.

DATA IN N8N IS A LIST, AND EVERY NODE RUNS ONCE PER ITEM

Data moves between nodes as an **array of items**. When a node receives three items, **it executes three times** and passes three results onward.

You do not write a loop. The loop is implicit in the data.

Notice the contrast with your Actor, where you wrote `for (const student of students)` yourself. Same result, opposite philosophy.

n8n hides the loop. The Actor shows it. That is the difference in one line, and it recurs at every level of the two platforms — right up to the agent loop in Chapter 6.

3.7 Which to reach for

Situation	Reach for
Something you will maintain for a semester	Actor
Something a colleague must be able to run	Actor
Something that runs on a schedule and emails a report	n8n, calling an Actor
Connecting to Gmail, Sheets, Slack, a calendar	n8n
Anything where you need to see exactly why it did what it did	Actor
Getting something working in twenty minutes	n8n

They are complements, not rivals. The most common real arrangement is n8n handling the schedule and the connections while an Actor does the work.

CHAPTER 3 — CHECK YOURSELF

- [] I can name the three sections of every Actor
- [] I know which two files I edit and which two I do not
- [] I can explain why prompts belong in the input schema
- [] I know why memory setting matters to my \$5
- [] I know to debug by finding the first wrong log line

→ **Lab Test 3A: Your first Actor, no AI** → **Lab Test 3B: The same thing in n8n**

Chapter 4 — Adding intelligence

4.1 What changes, and what it costs

Everything in Chapter 3 was deterministic. Same input, same output. Adding an AI call changes that permanently, and you should know exactly what you are trading.

	Deterministic code	An AI call
Same input twice	Identical output	Similar, not identical
Fails by	Throwing an error and stopping	Continuing, with confident nonsense
Cost per run	Nothing	API quota
Speed	Milliseconds	Seconds
Debugging	Read the error	Read the output and judge it

The second row is the one that matters, and every design decision in the rest of this book follows from it.

A broken deterministic Actor throws an error and stops. A broken AI Actor completes successfully and hands you something that looks correct.

4.2 Keys belong in environment variables

Your Gemini API key goes in the Actor's **Settings → Environment variables**, named `GEMINI_API_KEY`, with the **Secret** box ticked. Your code reads it as `process.env.GEMINI_API_KEY`.

TWO THINGS ABOUT THIS THAT CATCH EVERYONE

Environment variables are applied at build time. Add the key and run without rebuilding, and your Actor reports the key as missing. Add it, then build.

Never put a key in an input field. An input value is stored with the run and visible in the run record. A secret environment variable is encrypted and never shown. Form this habit here, where it costs nothing, so it is automatic when it matters.

4.3 Model choice is made for you

On the free tier the Pro models moved behind billing during 2026, so the free tier is a Flash-only tier.

This is less of a constraint than it sounds. Flash handles every exercise here comfortably. Use the exact model identifier your facilitator has written on the board — pinning one model across the room means everyone hits the same behaviour and problems become diagnosable.

Put the model name in the **input schema**, so a change does not require a rebuild.

4.4 Rate limits, and designing around them

Free-tier Flash allows roughly **15 requests per minute**, with a daily ceiling in the low thousands. Both are per-key, and everyone has their own key, so the room does not compete for one quota.

The failure looks like this. You write a loop over thirty students, each triggering one call. The first fifteen or so succeed and the rest fail with **HTTP 429**.

TWO RULES THAT PREVENT THIS

Test on three items, never thirty. Keep your input small while building. Scale up only when the code is finished and correct.

Sleep between calls. Every Actor in this manual has a 4.5-second delay in the loop for exactly this reason. Do not remove it. Slow and complete beats fast and half-failed.

Two distinct failures produce a 429, and it is worth telling them apart:

Error	Meaning	Recovery
Rate limit (RPM)	Too many requests this minute	Clears in under a minute
Quota exceeded (RPD)	Daily allowance gone	Does not clear today

If you hit the second during the workshop, say so rather than retrying.

4.5 One failure must not kill the run

Wrap each AI call in a try/catch **inside** the loop, and record the failure rather than throwing.

Without it, one malformed response from one student ends the run and you lose everyone else's results too. With it, the bad one is flagged and the rest complete.

This is not defensive over-engineering. You will hit a failed call today.

4.6 Getting structured output

Prose is hard to use downstream. If later code needs the score, you cannot reliably pull it out of a paragraph.

Ask for JSON explicitly, then parse it. And expect the model to wrap it in markdown fences despite being told not to:

```
function extractJson(raw) {
  const cleaned = raw.replace(/```json/gi, '').replace(/```/g, '').trim();
  const start = cleaned.indexOf('{');
  const end = cleaned.lastIndexOf('}');
  if (start === -1 || end === -1) {
    throw new Error(`No JSON object found: ${raw.slice(0, 200)}`);
  }
  return JSON.parse(cleaned.slice(start, end + 1));
}
```

Three lines of defence: strip the fences, find the outermost brace pair, and fail with the raw text visible so you can see what actually came back.

Put this in every parser you write. It eliminates the single most common cause of a failed run in this workshop.

4.7 Check that it followed your rule

Getting parseable JSON is not the same as getting correct JSON.

If your prompt says "set `needs_meeting` to true only if the percentage is strictly below 40", and a student scores 42%, the model will sometimes set it to true anyway — because it decided 42 is close enough to worry about.

That is not a crash. It is a plausible answer that quietly ignores your instruction, and it is the class of error you will spend the rest of this workshop learning to catch.

CHAPTER 4 — CHECK YOURSELF

- [] I understand that an AI call fails by continuing, not by stopping
- [] My key is a **secret environment variable**, and I know to rebuild after adding it
- [] I know to test on three items and to sleep between calls
- [] I can tell an RPM 429 from an RPD 429
- [] I know why the fence-stripping parser exists
- [] I know to check whether the model applied my rule, not just whether it returned JSON

→ Lab Test 4A: An Actor that calls Gemini → Lab Test 5A: Structured output and parsing

Chapter 5 — From a sequence to an agent

5.1 Where a fixed sequence stops

You now have an Actor that calls a model once per record. Consider something it cannot do.

A student asks: *"I did not understand the third normal form example from the class on 14 August. Can you explain it differently?"*

To answer this well, something must:

1. Work out which topic and which class this refers to
2. Retrieve your actual material for it
3. Notice the request is for a *different* explanation, and avoid repeating the original
4. Produce an explanation at the right level
5. If the student is still confused, try another approach

Steps 1 and 5 are the problem. In a fixed sequence you decide in advance what happens at every step. Here, what happens depends entirely on what the student said and how they respond.

That is the boundary. Crossing it needs three things.

5.2 The three additions

Tools

A tool is something the agent can *call*. A calculator. A lookup over your notes. A web fetch.

Critically: **you give the agent several tools and do not tell it which to use.** It reads the request and decides. That delegation of choice is the first thing that makes it an agent rather than a sequence.

Memory

A sequence forgets everything between steps. An agent must remember what it has already tried, or it will call the same tool repeatedly and get nowhere.

In the loop you will write, memory is simply an array called `trace` that grows each turn and gets pasted back into the next prompt.

The loop

The essential one:

```
think → act → observe → think → act → observe → ... → answer
```

It reasons about what to do, does it, looks at the result, and reasons again. It stops when it judges the task complete.

THE DEFINITION

A sequence is a path you drew.

An agent is a model choosing its own path, one step at a time, using tools you gave it.

5.3 What the loop actually looks like

The whole thing, stripped to its structure:

1. Build a prompt: instructions + tool list + history so far + the question
2. Ask the model what to do next
3. If it names a tool, run it and push the result into history
4. If it gives an answer, stop
5. Repeat until answered or the iteration cap is reached

Five steps. That is the entire idea, and you will write it in Chapter 6's lab as about sixty lines of ordinary JavaScript.

WHY WRITING IT YOURSELF IS WORTH THE EXTRA HOUR

n8n's Agent node does this. So do CrewAI, LangGraph and Mastra. All of them hide it behind configuration.

Configure one and it works, and you have learned an import statement. Write one and every framework afterwards is a matter of looking up method names, because they are all doing exactly this.

You will also be able to change the stopping condition, add a validation step, or log whatever you want — none of which is possible inside a node you cannot open.

5.4 The cost of the loop

Agents are worse than fixed sequences on every axis except flexibility, and you should know the bill before choosing one.

	Fixed sequence	Agent
API calls per run	1 per AI step	3 to 10+, unpredictable
Runtime	Seconds	Tens of seconds
Same input, same output	Roughly	No
Can loop forever	No	Yes, if badly configured
Debugging	Read the failing step	Read the whole reasoning trace
Free tier friendly	Yes	Marginal

THE ITERATION CAP IS NOT OPTIONAL

A misconfigured agent can loop indefinitely — calling a tool, disliking the result, calling it again. On a free tier this exhausts your daily quota in minutes.

Every agent you build takes `maxIterations` as an input. **Leave it at 5.** Set it before your first run, not after your first incident.

There is a matching instruction that prevents over-cautious looping: *"Once you have what you need, answer. Do not call more tools to double-check something you already have."* Without it, agents burn iterations on reassurance.

5.5 The design rule that matters most

GIVE THE AGENT FEWER TOOLS THAN YOU THINK IT NEEDS

An agent with three well-described tools substantially outperforms one with ten.

With too many it picks wrong, calls several unnecessarily, burns iterations deciding, and produces a trace you cannot debug.

Start with two. Add a third only when you have watched it genuinely fail for want of it.

The instinct is to give it every capability available. Resist it. Every tool you add makes every decision harder.

5.6 Choosing correctly

Return to Chapter 1's question and add a second:

1. Do the steps change depending on what I find? 2. Am I prepared to check the output every time?

An agent needs a **yes** to both. If the steps are fixed, write a sequence. If you would not check its work, do not deploy it — a sequence you trust beats an agent you do not.

5.7 What comes next

Day 3 builds four things of rising sophistication:

Build	Concept
First agent loop	Tools, memory, the loop, written by hand
Attainment agent	Tool use — it calculates rather than guesses
Grading pipeline	Multi-agent — a second agent catches the first one's errors
Doubt-solving agent	Grounding and guardrails

Each fixes a failure you will watch the previous one commit.

CHAPTER 5 — CHECK YOURSELF

- [] I can name the three things that turn a sequence into an agent
- [] I can recite the five-step loop structure
- [] I know what `maxIterations` does and what to set it to
- [] I know why fewer tools beat more
- [] I can state both questions to ask before choosing an agent

→ Lab Test 6A: Your first agent loop, written by hand

Chapter 6 — Agent architecture

6.1 Five components

Every agent, in every framework, in every language, is these five things. Frameworks differ in naming and syntax and in nothing else. Learn these once and every agent library you meet is a matter of looking up method names.

Component	Role	Get it wrong and
Model	The reasoning engine	Cheap models loop; expensive ones bankrupt you
Instructions	Who it is, what it must and must not do	It improvises, confidently
Tools	What it can actually do	It guesses instead of checking
Memory	What it retains across turns	It repeats itself and forgets context
Loop control	When to stop	It runs forever, or gives up too early

6.2 The loop, concretely

The pattern underneath nearly all agents is **reason, act, observe** — commonly called ReAct. It is much simpler than the literature suggests, and you will have written it yourself by the end of Lab 6A.

Given the question "*What is the CO3 attainment for my Semester 4 class?*", the trace runs roughly:

```

THOUGHT: I need the marks data. I have a tool called
         read_marks_data. I should call it.
ACTION:  read_marks_data()
OBSERVE: Received 20 rows with roll_no and q1 through q5.

THOUGHT: I need to know which questions map to C03.
         get_co_mapping will tell me.
ACTION:  get_co_mapping()
OBSERVE: C03 is assessed by q2 and q4. Threshold 0.6.

THOUGHT: Now I need the count above threshold. I must not
         estimate this. I will use the calculator.
ACTION:  calculate_attainment({questions:["q2","q4"],
                                threshold:0.6,
                                max_per_question:10})
OBSERVE: 11 of 20 students above threshold. 55.00%.

THOUGHT: I have everything. 55% is Level 1 on the institute
         scale. I can answer now.
ANSWER:  Using q2 and q4, threshold 60%, across 20 records:
         C03 attainment is 55.00% (11 of 20)...

```

Three things deserve attention.

The agent decided the sequence. You did not tell it to read the marks before the mapping. It worked that out.

It used a calculator for the arithmetic. Deliberately, and Chapter 7 explains why this is the difference between a usable answer and a plausible one.

It stopped when done. That judgement is fallible, which is why the iteration cap exists.

READ THE TRACE

In your Actor, every one of those lines appears in the run log, because you wrote the `console.log` statements that produce them.

Read it every time while you are learning. It is the only way to understand *why* an agent did something. An agent that produced a wrong answer usually shows you exactly where it went astray — a tool returned something unexpected, or the instructions were ambiguous at one specific decision.

This is one of the real advantages of writing the loop yourself: you control what gets logged. In a framework you see whatever it chose to expose.

6.3 Writing agent instructions

Agent instructions are prompts, so RCTFG still applies. Three things are added.

Tell it about its tools, and when to use them

Do not rely on tool names alone.

TOOLS AVAILABLE TO YOU:

- `read_marks_file`: Returns all student marks as rows. Call this first for any question about scores or attainment.
- `get_co_mapping`: Returns which questions assess which CO, and the institute's attainment thresholds. Call before any attainment calculation.
- `calculate_attainment`: Computes attainment percentage. You **MUST** use this for every calculation. Never compute percentages or counts yourself.

That last instruction is the most important line in the whole prompt. Models are willing to do arithmetic and bad at it. Forbid it explicitly.

Tell it when to stop

When you have the attainment figure and its level, answer immediately. Do not call additional tools to double-check a result you already have.

Without this, agents loop out of over-caution — re-reading files, recalculating, burning quota on reassurance.

Tell it what to do when stuck

If a tool returns an error or empty data, report exactly what you tried and what happened. Do not estimate the answer from partial data. Do not retry the same call more than twice.

THE FAILURE THIS PREVENTS

Without this instruction, an agent whose tool fails will frequently **fall back on estimation and present the estimate as a result**, with no indication that anything went wrong.

For a CO attainment figure headed into an NBA file, that is not a minor bug. It is a fabricated number in an accreditation document, produced by a system you told your department to trust.

Write this instruction into every agent you build.

6.4 Memory

In the loop you write, memory is an array. Each turn appends the thought, the tool called, the arguments and the result; the whole array is pasted into the next prompt.

That is the simplest workable form, and for everything in this workshop it is the right one.

Three kinds exist, and you rarely need the third:

Type	Holds	Use for
Full trace	Everything tried this run	Almost everything. Start here.
Windowed	Last N steps only	Long conversations that outgrow the context
Long-term	Facts retrievable across sessions	Rarely needed at our scale

MEMORY HAS A SIZE LIMIT YOU WILL EVENTUALLY HIT

Every turn, the whole trace goes into the prompt. A tool that returns a large result — a full marks table, a fetched web page — makes each subsequent prompt bigger.

The Actor code truncates each stored result to a fixed length for exactly this reason. If your agent starts failing on iteration four with a token error, that is why.

6.5 The single most useful design rule

If you take one rule from Day 3:

GIVE THE AGENT FEWER TOOLS THAN YOU THINK IT NEEDS

An agent with three well-described tools substantially outperforms one with ten.

With too many, it picks wrong, calls several unnecessarily, burns iterations deciding, and produces a confused reasoning trace you cannot debug.

Start with two. Add a third only when you have watched it genuinely fail for want of it.

The instinct is to give it every capability available. Resist it. Every tool you add makes every decision harder.

CHAPTER 6 — CHECK YOURSELF

- [] I can name the five components of any agent
- [] I can describe the think-act-observe loop
- [] I know why "never compute this yourself" belongs in the instructions
- [] I know why fewer tools beat more

Chapter 7 — Tools, and why they matter more than anything else

7.1 The demonstration you need to see

This will be run live. Please watch it rather than reading ahead, because the point lands better when you see the numbers.

Attempt one. Paste an entire marks table — sixty students, five questions each — into a prompt and ask for CO3 attainment where CO3 is assessed by questions 2 and 4 at a 60% threshold.

You will get a clean, confident, precise answer. Something like *"CO3 attainment is 72.4%, with 43 of 60 students meeting the threshold."*

Now verify it by hand. Count the rows yourself.

It will be wrong. Not wildly — close enough to be believable, which is precisely the danger. It will be wrong in a way that survives a glance and fails an audit.

7.2 Why it is wrong

An LLM does not compute. It predicts plausible text. Asked for an average of sixty numbers, it produces a number that *looks* like a plausible average of those numbers. Sometimes it is right. It is never reliable.

The failure has a specific and dangerous shape:

- It never says "I am not sure"
- The number is always plausible
- It is presented with the same confidence as a correct answer
- It changes if you ask again

WHY THIS MATTERS MORE IN ACADEMIC WORK THAN ALMOST ANYWHERE ELSE

A wrong attainment figure does not look wrong. It flows into a CO-PO matrix, into an NBA self-assessment report, into a departmental review. Nobody recalculates it by hand — that was the whole point of automating it.

By the time it is questioned, it has been in three documents for eight months.

Never let a model do arithmetic that matters.

7.3 The fix

Give the agent a calculator and forbid it from doing the maths itself.

```
// Tool: calculate_attainment
// Description given to the agent:
//   "Computes CO attainment. Provide the marks rows, the list of
//   question columns assessing the CO, and the threshold as a
//   decimal. Returns exact counts and percentage.
//   You MUST use this for all attainment calculations."

const rows      = $input.first().json.rows;
const questions = $input.first().json.questions;
const threshold = $input.first().json.threshold;
const maxPerQ   = $input.first().json.max_per_question;

const maxTotal = questions.length * maxPerQ;

let met = 0;
for (const row of rows) {
  let scored = 0;
  for (const q of questions) {
    scored += Number(row[q]) || 0;
  }
  if (scored / maxTotal >= threshold) met++;
}

return [{
  json: {
    students_assessed: rows.length,
    students_meeting_threshold: met,
    attainment_percentage: Number((met / rows.length * 100).toFixed(2)),
    questions_used: questions,
    threshold_applied: threshold
  }
}];
```

Now the agent decides *which* calculation to run and *how to interpret* the result — both things it is genuinely good at — while the arithmetic is done by arithmetic.

THE PRINCIPLE, GENERALISED

The model decides. The tool computes.

Anywhere a number, a date, a lookup or a fact must be exactly right, it comes from a tool. The model's job is choosing which tool, supplying the right arguments, and interpreting what comes back.

This one principle separates an agent you can put in front of a department from a demonstration.

7.4 Tool categories for academic work

Category	Examples	Why
Calculation	Attainment, statistics, grade aggregation	Models cannot be trusted with numbers
Retrieval	Course notes, syllabus, previous papers	Models do not know your material
External data	Apify Actors, web APIs	Models have a training cutoff
Action	Send email, write a sheet, create a file	Producing an effect, not just text
Validation	Check marks total, verify CO exists	Catching the agent's own errors

The last category is underused and valuable. A tool that returns "*Error: marks total 22, expected 20*" lets the agent detect and correct its own mistake inside the loop — before you ever see the output.

7.5 Writing good tool descriptions

The agent chooses tools by reading their descriptions. The description is a prompt, and vague ones cause bad choices.

Poor:

```
Name: calc
Description: Does calculations
```

Good:

```
Name: calculate_attainment
Description: Computes CO attainment percentage from student marks.
Requires: rows (array of student records), questions (array of
column names assessing this CO), threshold (decimal, e.g. 0.6),
max_per_question (integer).
Returns exact counts and percentage.
Use for ALL attainment calculations. Never calculate manually.
```

Three rules: say what it does, say exactly what it needs, and say when to use it.

7.6 The verification habit

A tool guarantees the *calculation* is right. It does not guarantee the agent passed the *right arguments*.

The agent might correctly call `calculate_attainment` while passing questions 2 and 5 when CO3 is actually assessed by 2 and 4. The maths is flawless. The answer is wrong.

MAKE THE AGENT STATE ITS INPUTS

Add to the instructions:

```
When reporting any calculated result, state the exact inputs
you used: which questions, which threshold, how many student
records. Do this before stating the result.
```

Output becomes:

"Using questions q2 and q4, threshold 60%, across 58 student records: CO3 attainment is 70.7%."

Now a wrong argument is **visible at a glance**. You will spot "q2 and q5" instantly. You would never have spotted it inside a correct-looking percentage.

This is the same trick as asking for CO justifications in Chapter 2, and it is just as effective here.

CHAPTER 7 — CHECK YOURSELF

- ☐ I have seen a model produce a confident, wrong calculation
- ☐ I can explain why it fails in that specific shape
- ☐ I can state the principle: the model decides, the tool computes
- ☐ I know why making the agent state its inputs is a safety feature

→ **Lab 7: Building the CO-PO attainment agent**

Chapter 8 — Multi-agent patterns

8.1 The problem one agent cannot solve

A single agent has one perspective. It cannot see its own blind spots, because the blind spot and the judge are the same thing.

Grading is where this becomes acute. Ask one agent to grade five submissions against a rubric and two errors appear reliably:

It rewards fluency. A confidently written answer that is wrong scores higher than a correct answer written awkwardly. This is a serious problem, and it disadvantages exactly the students who are already disadvantaged — those writing in a second or third language.

It drifts. Grade five in sequence and the standard shifts. The fifth is graded against a different bar than the first, because the earlier submissions have coloured its sense of what is typical.

Neither is fixable by better prompting. Both are fixable by a second agent.

8.2 Pattern one: generator and critic

The most useful pattern in academic work, by a distance.

```
[Generator] → [Critic] → decision → [Generator revises] or → [Output]
```

Two agents, different instructions, opposing incentives. The generator produces. The critic's *only* job is to find fault.

The critic works because its instructions do not ask it to be balanced:

You are a moderator reviewing a colleague's grading. Your job is to find problems, not to agree.

Check specifically:

1. Does the score match the rubric, criterion by criterion?
2. Has fluent writing been rewarded over correct content?
3. Has correct content been penalised for poor expression?
4. Is this graded to the same standard as the other submissions in this batch?
5. Is every score justified by something actually in the submission, rather than assumed?

For each criterion, state AGREE or DISAGREE with a reason. If you disagree, state the score you would award.

Do not soften your assessment to be agreeable. A moderator who agrees with everything provides no value.

That final paragraph matters. Without it, models default to agreeableness and you have paid twice for one opinion.

APPLICATIONS BEYOND GRADING

- Question paper generator + a reviewer checking CO coverage and marks totals
- Lesson plan generator + a reviewer checking it fits the allotted time
- Feedback writer + a reviewer checking tone is constructive rather than crushing
- Any document generator + a reviewer checking it against a template

Anywhere you would ask a colleague to look over your work, this pattern applies.

8.3 Pattern two: pipeline

Specialists in sequence, each doing one thing well.

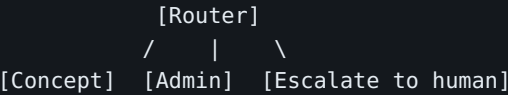
[Grader] → [Moderator] → [Feedback Writer]

Grading and feedback-writing are genuinely different tasks. Grading needs consistency and strictness. Feedback needs warmth and specificity. One agent asked to do both does both indifferently — feedback that reads like a mark sheet, or grading softened by the wish to be encouraging.

Separated, each does its own job properly.

8.4 Pattern three: router

One agent classifies, then dispatches.



For a student-facing agent: a conceptual doubt goes to the explainer, an administrative question about deadlines goes to a different handler, and anything about grades, extensions or personal difficulty is escalated to you.

THE ESCALATION PATH IS NOT OPTIONAL

Any student-facing agent must have a route to a human, and it must trigger on more than the obvious cases.

A student asking about a deadline extension may be telling you something about their circumstances. A student asking the same conceptual question for the fifth time is telling you something. A student whose message mentions personal difficulty needs a person, immediately.

Design the escalation path first, then the rest of the agent. Doing it the other way round means it gets bolted on badly or forgotten.

8.5 What this costs

Multi-agent is expensive, and you should choose it knowingly.

Design	Calls per submission	30 submissions
One grader	~1	~30
Grader + moderator	~2-3	~60-90
Grader + moderator + feedback	~4-6	~120-180

Against a free-tier daily allowance in the low thousands, the third design will process perhaps a few hundred submissions per day, assuming nothing else runs.

DESIGN FOR THE WORKSHOP, THEN DESIGN FOR REALITY

Every lab in this workshop uses **five sample submissions**, sequentially, with a delay. That is deliberate.

Before running anything against a real class, do the arithmetic in the table above against your actual numbers, and add a Wait node. A flow that works on five and dies on eighty is a flow that will fail at exactly the moment you needed it.

8.6 Where multi-agent is not worth it

Honestly: most of the time.

If a task has one right answer and a deterministic check exists, use the check. Verifying that a question paper totals 20 marks needs a line of JavaScript, not a reviewing agent.

Use a second agent only for judgements that genuinely require judgement. Was this graded fairly? Is this feedback constructive? Does this paper actually cover the outcomes it claims? Those need a second opinion. Arithmetic does not.

CHAPTER 8 — CHECK YOURSELF

- ☐ I can name the two specific failures a single grading agent commits
- ☐ I know why the critic's instructions must forbid agreeableness
- ☐ I have calculated the API cost of my intended design
- ☐ I know that an escalation path is designed first, not last

→ **Lab 8: Multi-agent rubric grading**

Chapter 9 — Apify: giving agents reach

9.1 What an Actor is

An **Actor** is a program that runs in Apify's cloud, takes structured input, and returns structured output. That is the whole idea.

It matters here for one reason: an agent's knowledge is frozen at its training cutoff and confined to what you paste in. An Actor lets it reach out and fetch what actually exists, now.

Component	What it is
Actor	The program
Run	One execution
Dataset	The structured results
Key-value store	Files and single values from a run
API token	Your credential — treat as a password

9.2 Academic uses that justify the effort

Be selective. Not every task needs web data.

Use case	Actor's job
Curriculum benchmarking	Fetch programme structures from peer institutes
Literature assistant	Collect recent paper abstracts on a topic
Course knowledge base	Gather documentation for a tool you teach
Affiliation compliance	Retrieve the current syllabus from the university site
Placement preparation	Collect current job descriptions to align course content

9.3 The cost model — read this before running anything

This section will save you real money and real frustration.

The free plan gives **\$5 of platform credit each month**, requires no credit card, and allows 25 concurrent runs. Credits reset monthly and **do not roll over**.

Two ways to spend it, and they differ by orders of magnitude.

Building your own Actor: essentially free

You pay only for compute, billed in **Compute Units** — 1 CU is 1 GB of RAM running for 1 hour. A lightweight Actor at 512 MB for 30 seconds costs a fraction of a cent. You can run your own Actors hundreds of times within the free credit.

Running Store Actors: expensive, fast

Here is the trap.

THE APIFY STORE PRICING TRAP

Store Actors advertise a headline rate like *"from \$2 per 1,000 results"*. The word **from** is doing heavy lifting: that is the best-tier rate, unlocked by a paid subscription.

The free plan sits in the FREE pricing tier and receives no discount, so you typically pay two to three times the headline number. This is billed per result, on top of compute.

Concretely: a location-scraping Actor at roughly \$0.0040 per result means your \$5 covers about 1,250 results. Then runs stop.

This is why the labs teach you to build Actors rather than rent them. A self-built Actor for your specific need costs a fraction of a cent per run. The equivalent Store Actor could consume your month's credit in a single afternoon.

There is no overage risk — when credit is exhausted, runs simply stop and no invoice appears. But your workshop exercise stops with it.

Data retention

Free-tier results are retained for **seven days**.

EXPORT BEFORE YOU LEAVE

Anything you build this week disappears from Apify by the middle of next week unless exported.

Every Apify lab ends with an export step. Please do not skip it. Faculty who intended to use their workshop output the following month have found it gone.

9.4 Connecting Apify to an agent

Two routes.

As a tool the agent calls. Add a fetch tool to your Actor's tools object, or an HTTP Request tool in n8n. The agent decides when to fetch. Use when the need is genuinely conditional.

As a step before the agent. Run the Actor first, feed results in as context. Use when you always need the data. Cheaper, faster, more predictable — **and usually the right choice.**

PREFER THE SECOND

The instinct is to make everything a tool the agent can call. Resist it.

If you always need the data, fetching it conditionally adds a decision the agent can get wrong, plus an extra API round-trip, for no benefit. Fetch it up front and hand it over.

Reserve tools for genuinely conditional needs.

9.5 Security

YOUR APIFY TOKEN ON A SHARED MACHINE

The API token grants full access to your Apify account.

- Never paste it into a shared document, email or chat
- Never commit it to a repository
- On Apify, store it as a **secret environment variable**, never in an input field where it is saved with the run
- In n8n, store it as a **credential**, never typed into a node parameter
- **Before leaving the lab on the final day, revoke the token** in Settings → Integrations and generate a fresh one for your own machine

Exported n8n workflow JSON can contain parameters. A token typed into a node rather than stored as a credential travels with any file you share.

CHAPTER 9 — CHECK YOURSELF

- [] I know the difference in cost between building and renting an Actor
- [] I know my free credit is \$5/month and does not roll over
- [] I know results vanish after seven days unless exported
- [] I know why "fetch before, not during" is usually right
- [] I know to revoke my token before leaving

→ **Lab 9: Building your first Apify Actor** → **Lab 10: Using an Actor as an agent tool**

Chapter 10 — Grounding an agent in your own material

10.1 The problem

Ask a general model about third normal form and you get a competent, generic explanation. Ask it about *the example you used in class on 14 August* and it invents one.

For a student-facing agent that is not a minor flaw. A confident explanation that contradicts your teaching is worse than no agent at all, because the student cannot tell which is authoritative.

Grounding means constraining an agent to answer from material you supply.

10.2 Two approaches

	Context stuffing	Vector retrieval
How	Put all material in the prompt	Index it; retrieve relevant pieces per query
Effort	Minutes	Hours
Material size	Up to a few hundred pages	Unlimited
Accuracy	Excellent when it fits	Good, depends on retrieval
Cost per call	Higher — you send everything	Lower

START WITH CONTEXT STUFFING, AND PROBABLY STAY THERE

Modern models have very large context windows. One course's material — syllabus, notes, worked examples, previous papers — usually fits comfortably.

Context stuffing is simpler, more accurate, and requires no infrastructure. **Most faculty never need to go further.**

Build the vector pipeline only when your material genuinely does not fit, or when you are grounding across an entire programme rather than one course. Reaching for it first is the most common way to spend three days building something a five-minute approach would have handled.

10.3 The three instructions that make grounding work

Supplying material is not enough. The agent must be told how to treat it.

You answer ONLY from the COURSE MATERIAL provided below.

1. If the answer is in the material, answer using it, and cite which section or unit it came from.
2. If the answer is NOT in the material, say exactly:
"That is not covered in our course material. Please check with your instructor." Do not answer from general knowledge.
3. If the material contradicts what you believe to be true, FOLLOW THE MATERIAL and note the discrepancy so the instructor can review it.

The third instruction is the one people leave out, and it matters. Your notation, your simplifying assumptions, your specific conventions may differ from the textbook consensus. The agent must follow *you*, and flag the difference rather than silently correcting you in front of a student.

10.4 The Socratic constraint

For a student-facing agent this is not a nicety. It determines whether your colleagues will accept the tool at all.

You do not give complete solutions to assessed problems.

For a homework or assignment question:

- Identify the concept being tested
- Ask what the student has tried so far
- Give the NEXT step only, not the full solution
- Point to the relevant section of the notes
- If they are stuck after two exchanges, give a worked example of a SIMILAR problem, never the assigned one

For conceptual questions not tied to an assessment, explain fully and freely.

WITHOUT THIS, YOU HAVE BUILT A HOMEWORK MACHINE

An agent trained on your course material that hands out complete solutions to assigned problems is not a teaching aid. It is a highly effective cheating tool that you personally built, deployed, and endorsed.

Faculty will reject it, correctly. **The Socratic constraint is part of the build, not a refinement to add later.**

10.5 The feature you will value most

Log every question students ask.

Not for surveillance — aggregate patterns, not individual tracking. Within two weeks you will know:

- Which topics generate the most confusion
- Which of your explanations are not landing
- What students ask at 11 pm the night before a deadline
- Which prerequisite knowledge is missing

Faculty who deploy these agents consistently report this as the most valuable output. The answers help students. **The questions tell you how to teach the course better next semester.**

CHAPTER 10 — CHECK YOURSELF

- ☐ I know why context stuffing is usually the right choice
- ☐ I can state all three grounding instructions
- ☐ I understand why "follow the material" over "follow the textbook" matters
- ☐ I have written a Socratic constraint for my own subject

→ **Lab 11: Grounding an agent in course material** → **Lab 12: The doubt-solving agent with guardrails**

Chapter 11 — Responsible use in an academic setting

This chapter is not a disclaimer. It contains the decisions that determine whether anything you build survives contact with your institution.

11.1 Student data

ANONYMISE BEFORE ANYTHING LEAVES YOUR MACHINE

Free-tier inputs and outputs may be used to improve the models. This is stated plainly in the terms.

For syllabi, question papers and lesson plans, that is unremarkable. For student data it is not acceptable.

Before any student data goes into any AI tool: - Replace names with roll numbers, or better, with sequential identifiers - Remove email addresses and phone numbers - Remove anything identifying in free-text responses - Keep the mapping file locally and never upload it

Every dataset in this workshop is already anonymised. Form the habit here.

11.2 Grading

Three questions to answer before any AI-assisted grading touches a real mark.

Does your institution permit it? Find out before, not after. Many institutions have no policy, which is not the same as permission — it means you are setting precedent, so set it deliberately and in writing.

Do students know? Our position is that they should. Undisclosed AI grading, discovered later, damages trust disproportionately to the benefit gained.

Who signs off? The agent proposes. **A human awards the mark.** Every time, without exception.

THE DEFENSIBLE POSITION

"AI-assisted first-pass grading, moderated by a second AI reviewer, with every mark reviewed and awarded by the instructor. Students are informed. Any student may request full human re-grading."

That is defensible to a student, to a department head, and to an accreditation panel.

"The AI graded it."

That is not.

11.3 Bias

Grading models reward fluent writing. This is measurable and consistent, and it means a well-expressed wrong answer can outscore a correct answer expressed awkwardly.

In an Indian engineering classroom, where many students are working in their second or third language, this maps directly onto an existing inequity — and automating it means applying it uniformly, at scale, with the appearance of objectivity.

Mitigations, and none is sufficient alone:

- A moderator agent instructed specifically to check for this — the reason criterion 3 exists in Chapter 8's critic
- Rubrics that separate content from expression, scored independently
- Spot-check the bottom decile by hand, every time
- Compare AI-suggested marks against your own on a sample before trusting the system

11.4 Accreditation documents

Anything AI-generated that goes into an NBA or NAAC submission must be verifiable.

- **Attainment figures come from tools, never from a model's estimate** — Chapter 7
- Keep the raw data and the calculation, not just the output
- Anyone signing the document must be able to reproduce the numbers
- Say clearly what was AI-assisted if asked

THE SCENARIO TO AVOID

An attainment figure, generated in an afternoon, believed because it looked reasonable, entered into a self-assessment report, and questioned eighteen months later during a panel visit — with nobody able to reproduce it.

This is entirely avoidable, and it is avoided by using tools for arithmetic and keeping the working.

11.5 Detection

Detecting AI-generated student work is unreliable. Detectors produce false positives, and they produce them disproportionately for students writing in a second language.

Do not build an agent to catch AI use. It will be wrong, and it will be wrong about the students least able to contest it.

Design assessments that are robust instead: in-person components, viva, iterative drafts, process over product, problems grounded in material specific to your class.

11.6 What to tell students

Plainly, early, in writing:

- What AI tools are used in the course, and for what
- Whether AI-assisted grading is used, and how to request human re-grading
- What they may and may not use AI for in their own work
- That any course agent logs questions in aggregate to improve teaching

CHAPTER 11 — CHECK YOURSELF

- ☐ I have anonymised any real data before using it
- ☐ I know my institution's position on AI-assisted grading, or know I need to ask
- ☐ I can state the defensible grading position in one sentence
- ☐ I understand the fluency bias and who it disadvantages
- ☐ I will not build a detection agent

Chapter 12 — From workshop to practice

12.1 Why most of this will not survive

Honestly: most faculty leave workshops like this energised, and build nothing. Two reasons.

They start too big. Back at work with a full teaching load, "build an AI teaching ecosystem" competes with grading, and loses.

Nothing was already in progress. A thing half-built gets finished. A thing not started does not get started.

The remedy for both is the same: **finish something small this week.**

12.2 This week

Pick one thing that saves you time on work you are already doing:

If you are about to	Build
Set an internal assessment	The question paper generator, on your actual next paper
Compute attainment	The attainment agent, on last semester's real marks
Grade a batch	The grader, on five you have already graded, and compare
Answer the same doubt repeatedly	The doubt agent, for one unit only

The comparison in row three is the highest-value exercise available to you. Grade five yourself, run the agent, compare. You learn precisely where it agrees, where it diverges, and how far to trust it — on your own material, with a ground truth you produced.

12.3 The first month

Weeks 1-2: one workflow, one course. Use it in earnest. Note every time it fails.

Weeks 3-4: fix what failed. Show one colleague. Their questions will find the assumptions you cannot see.

Only then consider a second use case.

12.4 At department level

Individual use scales badly. If this is to become departmental, three things are needed.

A shared library. One repository of workflow exports, prompt templates and rubrics. Otherwise every colleague rebuilds the same question paper generator, slightly worse.

A policy, written down. What may be AI-assisted, what must be human, what students are told, how data is handled. Written before the first incident, not after.

Someone who owns it. Not a committee. One person who maintains the shared library and is the first call when something breaks. Without this, it decays within a semester.

12.5 Cost, honestly

Everything in this workshop runs free. That holds for individual use and stops holding at department scale.

Scale	Realistic position
One teacher, own courses	Free tier is genuinely sufficient
A department piloting	Free tier plus occasional paid runs
Student-facing, whole cohort	Paid tier required. Budget it properly.

Two things worth raising with your department:

Apify Creator Plan — \$1/month with a one-time \$500 platform credit to spend within six months. For faculty building their own Actors, this transforms what is affordable.

Apify for Universities — typically around 50% off paid plans for verified academic use.

Both change the economics enough to be worth an email.

12.6 What to expect in two years

Brief, because forecasts age badly.

Models get cheaper and more capable. Agent frameworks consolidate. Institutional policy tightens — probably faster than most departments expect.

The durable skill is not any tool in this book. It is knowing **what to automate, what to keep human, and how to tell the difference**. That judgement transfers to whatever replaces n8n and Gemini. The specific tools will not.

12.7 One thing, by Friday

Write it down now, before you leave.

Concrete: *"Run the question paper generator for my Unit 3 internal on Thursday."*

Not: *"Explore AI in my teaching."*

The first gets done.

BEFORE YOU LEAVE

- ☐ I have copied every Actor's code to a file I hold, and exported any n8n workflows
 - ☐ I have saved my Apify results (they vanish in seven days)
 - ☐ I have revoked the API keys I used on this shared machine
 - ☐ I have written down one specific thing I will do by Friday
 - ☐ I have one colleague's name to show it to
-

Appendix A — Prompt templates

A.1 Question paper generator

You are an experienced professor of [SUBJECT] setting [ASSESSMENT TYPE] for a [YEAR] [PROGRAMME] course at an Indian engineering institute.

COURSE: [NAME AND CODE], Semester [N]

SYLLABUS [UNIT]:
[PASTE SYLLABUS TEXT]

COURSE OUTCOMES:
[C01: ...]
[C02: ...]

NOT TAUGHT IN THIS COURSE: [LIST OUT-OF-SCOPE TOPICS]

TASK:
Generate a [TOTAL] mark assessment covering [COs].
[MARKS BREAKDOWN BY BLOOM'S LEVEL]

BLOOM'S TAGGING RULES:
Remember - define, list, state, name
Understand - explain, describe, illustrate, classify
Apply - solve, compute, demonstrate
Analyse - compare, differentiate, decompose
Evaluate - justify, critique, assess
Create - design, construct, formulate
Tag on the verb the question requires, not topic difficulty.

OUTPUT FORMAT:
Markdown table: Q.No | Question | Marks | CO | Bloom's | Answer Outline
Then a "Marking Scheme" section, mark by mark.
For each question, one sentence justifying the CO and Bloom's tag.
No preamble.

CONSTRAINTS:
- Only syllabus topics above. Out-of-scope requests: say so, generate nothing.
- Every question tagged to a listed CO.
- Marks total exactly [TOTAL].

A.2 Moderator / critic

You are a moderator reviewing a colleague's [grading / question paper / lesson plan]. Your job is to find problems, not to agree.

Check specifically:

1. [Domain-specific criterion]
2. [Domain-specific criterion]
3. Has fluent presentation been rewarded over correct content?
4. Is this consistent with the other items in this batch?
5. Is every judgement supported by something actually present?

For each criterion state AGREE or DISAGREE with a reason.
Where you disagree, state what you would do instead.

Do not soften your assessment to be agreeable. A moderator who agrees with everything provides no value.

A.3 Grounded course agent

You are a teaching assistant for [COURSE], [PROGRAMME], [INSTITUTE].

You answer ONLY from the COURSE MATERIAL below.

1. If the answer is in the material, use it and cite the unit or section.
2. If it is NOT in the material, say: "That is not covered in our course material. Please check with your instructor."
Do not answer from general knowledge.
3. If the material contradicts what you believe to be true, FOLLOW THE MATERIAL and note the discrepancy.

SOCRATIC CONSTRAINT:

For homework or assignment questions:

- Identify the concept being tested
- Ask what the student has tried
- Give the NEXT step only
- Point to the relevant notes
- After two exchanges, give a worked example of a SIMILAR problem, never the assigned one

For conceptual questions not tied to assessment: explain fully.

ESCALATE TO THE INSTRUCTOR if the student:

- Asks about marks, grades, extensions or deadlines
- Mentions personal difficulty of any kind
- Asks the same conceptual question a third time
- Expresses significant distress

Escalation message: "This is something to discuss with [INSTRUCTOR NAME] directly. Please email [ADDRESS]."

COURSE MATERIAL:

[PASTE]

A.4 Attainment agent

You are an academic data assistant computing CO attainment for [INSTITUTE].

TOOLS:

- `read_marks_file`: Returns all student marks. Call first for any question about scores.
- `get_co_mapping`: Returns which questions assess which CO and the institute thresholds. Call before any calculation.
- `calculate_attainment`: Computes attainment. You **MUST** use this for every calculation. **NEVER** compute percentages yourself.

RULES:

- Never perform arithmetic yourself. Always use the tool.
- Before stating a result, state the exact inputs used: which questions, which threshold, how many records.
- If a tool errors or returns empty data, report exactly what you tried and what happened. Do **NOT** estimate from partial data. Do not retry the same call more than twice.
- When you have the figure and its level, answer immediately. Do not re-verify a result you already have.

OUTPUT:

1. Inputs used
2. Attainment percentage and student counts
3. Attainment level per the institute scale
4. One sentence of interpretation

Appendix B — Troubleshooting

Symptom	Likely cause	Fix
<code>GEMINI_API_KEY is not set</code>	Added but not rebuilt	Rebuild. Env vars apply at build time.
Node/run red, <code>429</code>	Rate limit	Wait 60s; cut to 3 items; raise the delay
<code>429</code> persisting all day	Daily quota exhausted	Will not clear today. Tell the facilitator.
<code>No JSON object found</code>	Model returned prose or fences	Use the fence-stripping parser from 4.6
Agent loops to the cap	No stopping instruction	Add "once you have what you need, answer"
Agent ignores a tool	Description too vague	Rewrite: what it does, what it needs, when to use it
Agent invents numbers	Not forced to use the calculator	Add "NEVER compute yourself" — no carve-outs
Prompt shows literal <code>{{name}}</code>	Field name mismatch	Check spelling against your records
Actor build fails immediately	JSON syntax error in input schema	Check commas and brackets
<code>Cannot use import statement</code>	<code>package.json</code> missing <code>"type": "module"</code>	Add it
Apify run stops mid-way	\$5 monthly credit exhausted, or memory set too high	Check Console billing; lower memory
Apify results gone	Seven-day free-tier retention	Export immediately after every run
Token error on iteration 4	Trace grew too large	Truncate stored tool results
n8n expression shows <code>undefined</code>	Wrong field path	Check the previous node's OUTPUT, JSON tab
n8n Python Code node errors on venv	Known limitation of this install	Use the JavaScript Code node
n8n workflow vanished	Different profile, or machine restored	Always export to file before leaving

Appendix C — Glossary

Term	Meaning
Actor	A program running in Apify's cloud that takes input and returns structured data
Agent	A model that chooses actions, observes results, and decides the next step
Bloom's level	Cognitive level a question demands, from Remember to Create
CO / PO	Course Outcome / Programme Outcome
Compute Unit (CU)	Apify billing unit: 1 GB RAM for 1 hour
Context stuffing	Putting all reference material directly into the prompt
Credential	Stored secret in n8n, kept out of the workflow file
Expression	Live reference to data in n8n, written <code>{{ }}</code>
Actor	A program on Apify: structured input, work, structured output
Input schema	The file defining an Actor's input form — the fast path to change
Compute Unit	Apify billing unit: 1 GB RAM for 1 hour
Grounding	Constraining an agent to answer only from supplied material
Item	One JSON object flowing between n8n nodes
maxIterations	Cap on an agent's loop count. Always 5 here.
Node	One step in an n8n workflow
RCTFG	Role, Context, Task, Format, Guardrails
ReAct	The reason-act-observe agent loop
RPM / RPD	Requests per minute / per day
Tool	A function an agent can call to do something it cannot do itself
Trigger	The node that starts an n8n workflow